Universidade Federal de Minas Gerais (UFMG)

Fundamentos de Inteligência Artificial

Trabalho Prático 2: *Busca Local* e *Algoritmo Genético*

Prof. Cristiano Leite de Castro | Prof. João Pedro Campos

Instruções Gerais

Objetivo

O objetivo deste trabalho é comparar heurísticas de Busca Local e uma metaheurística populacional, o Algoritmo Genético, para resolução de um problema prático de Engenharia, com instâncias de três tamanhos diferentes, e observar, na prática, o equilíbrio entre **exploração** e **intensificação** discutido em aula.

Grupos e identificação

O trabalho pode ser feito em grupos de **no máximo 4 (quatro) alunos**. Grupos do trabalho anterior podem ser mantidos.

O que deve ser entregue

1. **Relatório** em PDF (sugestão: 5 páginas, sem contar os anexos). O código *não* deve ser colado no corpo do relatório.
2. **Código-fonte** em arquivo separado (um `.zip` ou um repositório Git), acompanhado de um `README` com as instruções exatas para reproduzir cada tabela e cada figura do relatório.

Vale mais um relatório que *explica honestamente um resultado ruim* do que um relatório que apresenta números bons sem entender de onde vieram. Cada grupo poderá ser arguido oralmente sobre qualquer trecho do código.

1 O Problema: Sequenciamento de Manutenção de Ativos

1.1 Contexto operacional

Uma concessionária de distribuição de energia precisa executar a manutenção preventiva de n ativos (religadores, transformadores de distribuição, chaves seccionadoras) espalhados pela área de concessão. Há uma **única equipe**, que parte da base no início do turno e atende um ativo de cada vez.

Para cada ativo j conhece-se: a posição (x_j, y_j), a duração do serviço p_j (em horas), o **prazo regulatório** d_j (hora-limite para a conclusão) e um **peso de criticidade** w_j, que mede o custo por hora de atraso — um alimentador que serve um hospital pesa mais do que um ramal rural.

O deslocamento entre dois pontos i e j consome

$$t_{ij} \;=\; \mathrm{round}\!\left(\sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}\right) \;\text{horas}, \tag{1}$$

com arredondamento para o inteiro mais próximo. A base é o ponto de índice 0, com $p_0 = 0$. A equipe **não** precisa retornar à base.

1.2 Modelagem

Assim como no sequenciamento de corridas na aciaria visto em aula, a solução candidata é uma **permutação** $\pi = (\pi_1, \pi_2, \ldots, \pi_n)$ dos n ativos: a ordem de visita. O instante de conclusão do k-ésimo ativo visitado é definido recursivamente por

$$C_{\pi_k} \;=\; C_{\pi_{k-1}} \;+\; t_{\pi_{k-1}\,\pi_k} \;+\; p_{\pi_k}, \qquad \text{com } \pi_0 = 0 \text{ e } C_{\pi_0} = 0. \tag{2}$$

A **função de avaliação** (*fitness*) a ser **minimizada** é o atraso ponderado total:

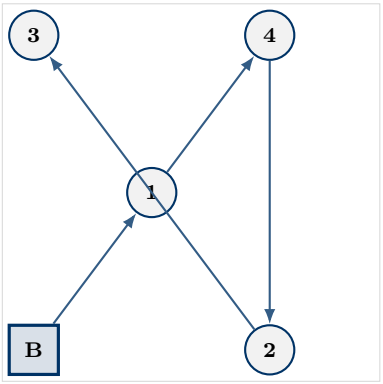
$$h(\pi) \;=\; \sum_{j=1}^{n} w_j \cdot \max(0,\; C_j - d_j)\,. \tag{3}$$

Trata-se de um problema de **otimização combinatória**: o espaço de busca tem $n!$ soluções possíveis. Para a instância grande deste trabalho ($n = 150$), isso é um número com mais de 260 algarismos — não há método sistemático que o percorra. Observe ainda duas propriedades úteis:

- **Toda permutação é viável.** Não há restrições rígidas (*hard constraints*) a satisfazer; o que existe é uma função de custo a minimizar. Na terminologia do Cap. 4, estamos diante de um problema de *soft constraints*, e não de satisfação.
- **h é não decrescente ao longo do prefixo da permutação.** O custo acumulado dos primeiros k ativos nunca diminui ao acrescentarmos o $(k{+}1)$-ésimo.

A dificuldade está na **tensão entre dois critérios gulosos antagônicos**: ir ao ativo *mais próximo* (que economiza deslocamento) ou ao ativo *mais urgente* (que evita multa). Nenhum dos dois, isoladamente, produz boas soluções.

Rota $\pi = (1, 4, 2, 3)$



base em $(0, 0)$; escala em horas de deslocamento

Cronograma resultante

ativo	p_j	d_j	w_j	C_j	$w_j(C_j - d_j)^+$
1	2	10	3	7	0
4	4	20	5	16	0
2	3	14	1	27	13
3	1	25	2	38	26

$h(\pi) = 39$

ótimo desta instância: $h = 22$

Figura 1: Exemplo ilustrativo com $n = 4$ e base em $(0, 0)$. A rota mostrada é a que um guloso por proximidade produziria, e custa 39. O ótimo é $\pi^\star = (1, 4, 3, 2)$, com custo 22: compensa adiar o ativo 2, que tem a menor criticidade, mesmo pagando um deslocamento maior. Verifique esses dois valores na sua implementação.

1.3 Formato das instâncias

Cada arquivo `manutencao_*.txt` tem a estrutura:

```
n
x_0  y_0                    (coordenadas da base)
x_1  y_1  p_1  d_1  w_1     (n linhas, uma por ativo, na ordem 1..n)
...
x_n  y_n  p_n  d_n  w_n
```

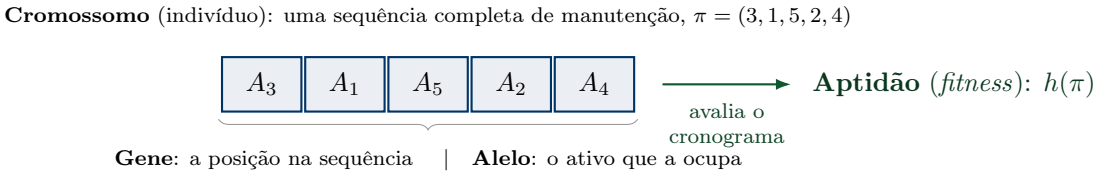


Figura 2: Mapeamento entre a terminologia biológica do *Genetic Algorithm* e os elementos deste problema, no mesmo formato visto em aula para o sequenciamento de corridas na aciaria. Como h mede um custo, o algoritmo procura o cromossomo de *menor* aptidão.

Os tempos de deslocamento **não** vêm no arquivo: calcule-os com a Equação (1). O arredondamento importa — sem ele, os valores de *fitness* de grupos diferentes não serão comparáveis. As instâncias fornecidas são:

Arquivo	n	$\sum_j p_j$	**observação**
`manutencao_pequena.txt`	12	55	ótimo global conhecido (ver abaixo)
`manutencao_media.txt`	50	239	referência: ver Tabela 1
`manutencao_grande.txt`	150	684	referência: ver Tabela 1

1.4 Verificação da sua implementação

Antes de rodar qualquer algoritmo, valide a sua função de *fitness*. Para `manutencao_pequena.txt`, a permutação identidade $\pi = (1, 2, 3, \ldots, 12)$ deve produzir

$$h(\pi) = 4366.$$

O **ótimo global** dessa instância, obtido por busca exata, vale $h^\star = 1875$. Para calibrar sua intuição: uma heurística gulosa razoável (escolher sempre o ativo que gera o menor acréscimo imediato de atraso) alcança 3168 — ou seja, 69% acima do ótimo. Há bastante espaço para os algoritmos trabalharem. Se a sua função de avaliação não reproduzir os dois números acima, nada do que vier depois fará sentido.

Tabela 1: Valores de referência. O ótimo da instância pequena foi obtido por busca exata. Os demais são os melhores valores encontrados pelos professores usando um orçamento muito maior do que o deste trabalho; servem como alvo, não como meta obrigatória.

Instância	**Valor de referência de h**	**Natureza**
pequena ($n = 12$)	1 875	ótimo global (busca exata)
intermediária ($n = 50$)	16 332	melhor valor conhecido
grande ($n = 150$)	60 417	melhor valor conhecido

2 Os três algoritmos

2.1 Arquitetura exigida: algoritmo ≠ problema

Os três algoritmos são genéricos; o problema é que é específico. Organize o código de modo que cada algoritmo **não conheça** o problema que está resolvendo, interagindo com ele apenas através de uma interface comum. Uma sugestão mínima:

Método	Papel
`solucao_aleatoria()`	gera um ponto de partida (uma permutação)
`h(s)`	função de avaliação (*fitness*), sempre de minimização
`sucessores(s)`	itera sobre todos os sucessores de s
`sucessor_aleatorio(s)`	sorteia um único sucessor de s
`cruzamento(s1, s2)`	operador de recombinação
`mutacao(s)`	operador de mutação

Essa separação não é preciosismo de engenharia de software: ela é o que torna viável rodar 9 combinações (3 algoritmos $\times$ 3 instâncias) e é exatamente o que permite responder à pergunta central do trabalho, que é sobre o *casamento* entre o algoritmo e a estrutura do problema.

2.2 Operadores de vizinhança (geração de sucessores)

Os três algoritmos precisam gerar sucessores de uma permutação. Implemente os três operadores clássicos da Figura 3 e informe, no relatório, o **tamanho da vizinhança** que cada um induz em função de n.

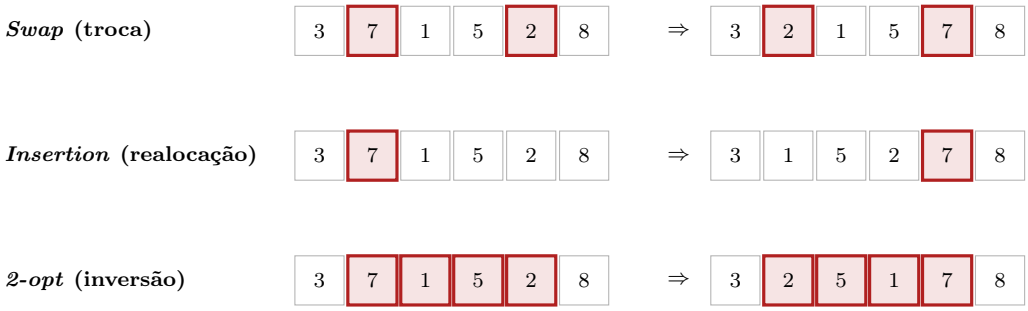


Figura 3: Os três operadores clássicos de geração de sucessores sobre permutações. Eles *não* são equivalentes: cada um induz um conjunto de sucessores diferente e, portanto, uma *superfície adaptativa* diferente para o mesmo problema.

2.3 (A) *Greedy Descent* com *Random Restart*

Implemente a descida gulosa (*greedy descent*, também chamada *best improvement* em aula): a partir de uma solução inicial, mova-se sempre para o sucessor de menor h, até atingir um **ótimo local** — isto é, uma permutação para a qual nenhum sucessor melhora a função de avaliação. Ao atingir o ótimo local, aplique ***random restart***: recomece de uma nova permutação aleatória, guardando sempre a melhor solução já vista, até esgotar o orçamento.

Vocês devem implementar e comparar pelo menos **duas** das três variantes de seleção do próximo sucessor discutidas em aula:

- ***Best improvement***: varre toda a vizinhança e escolhe o melhor sucessor;
- ***First improvement***: aceita o primeiro sucessor que melhora h;
- **Sucessor aleatório com aceitação de melhora**: sorteia um único sucessor e o aceita apenas se melhorar h.

A escolha entre elas tem consequências fortes sob orçamento fixo: na instância grande, uma única varredura completa da vizinhança de *swap* já consome $n(n-1)/2 = 11\,175$ avaliações. Reporte quantos *passos de melhoria* cada variante consegue dar dentro do orçamento e discuta o que isso significa.

2.4 (B) *Simulated Annealing*

Implemente o *simulated annealing* exatamente como visto em aula. Seja π a solução corrente e π' um sucessor sorteado ao acaso:

- se $h(\pi') \leq h(\pi)$, aceite π';
- caso contrário, aceite π' com probabilidade $e^{-(h(\pi')-h(\pi))/T}$ (distribuição de Gibbs).

Vocês devem:

- definir e justificar o ***annealing schedule*** adotado (por exemplo, o resfriamento geométrico visto em aula, $T \leftarrow \alpha T$), explicando como T_0, T_f e α foram calibrados. Atenção: a escala adequada de T depende da magnitude típica de $h(\pi') - h(\pi)$, que neste problema é da ordem de dezenas a milhares e *muda com o tamanho da instância* — uma temperatura calibrada na instância pequena não serve para a grande;
- registrar a **taxa de aceitação** de movimentos piores ao longo da execução e apresentá-la em um gráfico. Como visto em aula, uma têmpera bem calibrada começa praticamente como *random walk* (aceitando quase tudo) e termina praticamente como *greedy descent* (não aceitando nenhuma piora).

2.5 (C) *Genetic Algorithm*

Implemente um *genetic algorithm* com população de tamanho N, operadores de seleção, cruzamento com probabilidade P_c, mutação com probabilidade P_m e elitismo, seguindo o ciclo de vida e o pseudocódigo vistos em aula.

- **Inicialização**: amostragem aleatória de N sequenciamentos.
- **Seleção**: por torneio ou proporcional à aptidão (roleta). Lembre-se de que o problema é de **minimização**: explique no relatório como você converteu o custo h em uma medida de aptidão adequada ao operador de seleção escolhido.
- **Cruzamento**: como a codificação é uma permutação, os cruzamentos clássicos de um ou múltiplos pontos **duplicariam ativos** e produziriam cromossomos inválidos. Use um operador que preserve a validade da permutação — o *Order Crossover* (OX) visto em aula, o *Partially Mapped Crossover* (PMX) ou o cruzamento por arestas. Descreva o operador escolhido com um exemplo numérico passo a passo.
- **Mutação**: aplique uma pequena taxa de mutação para inserir diversidade e favorecer a exploração. A *swap mutation* vista em aula é o ponto de partida natural; outros operadores da Figura 3 também são admissíveis.
- **Critério de parada**: neste trabalho, o critério é o esgotamento do orçamento de avaliações (Seção 3), de modo que a comparação entre os três algoritmos seja justa.

Relate também a evolução da **diversidade** da população ao longo das gerações (por exemplo, o número de cromossomos distintos, ou a distância média entre pares de indivíduos). A convergência prematura — toda a população colapsando sobre um mesmo ótimo local — é o modo de falha mais comum do *genetic algorithm* e precisa ser diagnosticada com dados, não suposta.

3 Protocolo experimental (obrigatório)

Orçamento comum. A comparação só é justa se todos os algoritmos gastarem o mesmo recurso. O recurso contabilizado é o **número de avaliações da função** h: cada sucessor examinado pela *local search* conta 1, cada indivíduo avaliado pelo *genetic algorithm* conta 1. Use os orçamentos da tabela abaixo e *pare exatamente* ao atingi-los.

Instância	Orçamento B (avaliações)	Execuções independentes
pequena ($n = 12$)	50 000	30
intermediária ($n = 50$)	150 000	30
grande ($n = 150$)	300 000	10

Avaliação incremental. Avaliar uma permutação do zero custa $O(n)$ operações. Avaliar o *efeito* de um movimento costuma custar menos, porque apenas o sufixo da sequência é afetado. Implemente a avaliação incremental em pelo menos uma das vizinhanças, verifique com um teste automático que ela concorda com a avaliação completa, e relate o ganho de tempo medido. Isso *não* altera a contagem de avaliações — altera o tempo de parede, que também deve ser reportado.

Estatística. Para cada combinação algoritmo × instância, reporte média, desvio-padrão, mínimo e mediana do melhor h encontrado. Apresente *boxplots* comparando os três algoritmos em cada instância. Ao afirmar que um algoritmo é melhor que outro, sustente a afirmação com um teste não paramétrico (por exemplo, Mann–Whitney/Wilcoxon) e informe o p-valor. Médias próximas com dispersões grandes não autorizam conclusão alguma.

Curvas de convergência. Para cada instância, apresente um gráfico com o melhor h encontrado (eixo y) em função do número de avaliações gastas (eixo x), com uma curva por algoritmo (mediana das execuções). Para o *genetic algorithm*, acrescente também a curva da **média da população**, como visto em aula: a distância entre as duas curvas é o indicador visual direto da perda de diversidade.

4 Tarefas

T1 — Função de avaliação. Implemente $h(\pi)$ conforme a Equação (3) e valide-a com os dois valores de verificação da Seção 1. Mostre no relatório a saída do seu programa para esses dois casos.

T2 — Comparação das vizinhanças. Implemente os três operadores da Figura 3. Usando *greedy descent* com *random restart* e um mesmo orçamento, compare as três vizinhanças nas três instâncias: qual delas conduz a melhores ótimos locais? Qual delas é mais barata por movimento quando se usa avaliação incremental? Justifique a vizinhança que você adotará nos demais experimentos.

T3 — Experimentos. Rode os três algoritmos nas três instâncias seguindo integralmente o protocolo da Seção 3.

5 Análise comparativa (parte central do relatório)

O relatório deve conter, no mínimo:

1. **Tabela principal** com as 9 combinações (3 instâncias × 3 algoritmos), reportando média ± desvio-padrão e melhor valor de h. Na instância pequena, inclua a coluna de *gap* relativo ao ótimo conhecido, gap $= (h - h^\star)/h^\star$.
2. **Boxplots** por instância, comparando os três algoritmos.
3. **Curvas de convergência**, conforme a Seção 3.
4. **Testes estatísticos** para cada comparação par a par que vocês afirmarem no texto.
5. **Tempo** médio por execução, lado a lado com o número de avaliações, discutindo a diferença entre as duas métricas.

A discussão escrita deve, obrigatoriamente, abordar os seguintes pontos:

- **Ótimos locais.** Quantos ótimos locais distintos o *greedy descent* encontrou na instância pequena ao longo dos *random restarts*? Em quantas execuções ele alcançou $h^\star = 1875$? Compare com os outros dois algoritmos e relacione o resultado com a discussão de ótimo local × ótimo global vista em aula.
- **Exploração × intensificação.** Para cada algoritmo, aponte qual mecanismo cumpre o papel de exploração e qual cumpre o de intensificação, e mostre nos seus dados onde o equilíbrio foi bem ou mal ajustado.
- **Efeito da escala.** O *ranking* dos três algoritmos muda da instância pequena para a grande? Em caso afirmativo, formule uma hipótese sobre a causa, ligada ao custo de varrer a vizinhança e ao tamanho do espaço de busca. Em caso negativo, mostre que a estabilidade do *ranking* é estatisticamente sustentada.
- **Quando o *genetic algorithm* compensa?** Existe alguma instância em que ele supera, de forma estatisticamente significativa, as duas abordagens de *local search*? Formule uma hipótese em termos da codificação adotada — a recombinação consegue combinar bons trechos de sequência de dois pais em um filho melhor? — e proponha, execute e reporte **um** experimento adicional que teste essa hipótese.

6 Dicas e armadilhas comuns

- **Teste a função de avaliação primeiro.** Os dois valores de verificação da Seção 1 existem para isso. Metade dos erros graves em trabalhos desta natureza está na função de *fitness*, não no algoritmo.
- **Cuidado com a contagem de avaliações.** Se o *greedy descent* recalcula h da solução corrente a cada passo, isso também conta. Instrumente a contagem em um único lugar do código.
- ***Simulated annealing* mal calibrado vira outro algoritmo**: com T alta demais ele é um *random walk*; com T baixa demais, um *greedy descent*. O gráfico de taxa de aceitação revela imediatamente qual dos dois casos ocorreu.
- ***Genetic algorithm* sem diversidade é uma *local search* cara.** Se a população converge nas primeiras gerações, o restante do orçamento é desperdiçado. Meça a diversidade.
- **Guarde tudo em disco.** Rodar 9 combinações × dezenas de execuções e perder os dados por falta de um `.csv` é um erro caro de cometer na véspera da entrega.

Leituras de apoio

- Poole, D.; Mackworth, A. *Artificial Intelligence: Foundations of Computational Agents*. 3. ed. Cambridge University Press, 2023. Capítulo 4 — em especial as seções sobre *local search*, *random restart*, *random steps* e *simulated annealing*.
- Eiben, A. E.; Smith, J. E. *Introduction to Evolutionary Computing*. Springer-Verlag, 2003.
- Gaspar-Cunha, A.; Takahashi, R. H. C.; Antunes, C. H. (Orgs.). *Manual de Computação Evolutiva e Metaheurística*. Editora UFMG, 2013.
- Russell, S.; Norvig, P. *Artificial Intelligence: A Modern Approach*. 3. ed. Pearson, 2016. Capítulo 4.